International Journal of Control, Automation, and Systems VV(X) (YYYY) 1-6
http://dx.doi.org/10.1007/s12555-xxx-xxxx-x
ISSN:1598-6446 eISSN:2005-4092
http://www.springer.com/12555

Event-Driven Language Memory for Long-Horizon Vision-Language-Action Policies

[Author names]*

Abstract: Vision-language-action (VLA) policies act on the current observation and therefore fail on manipulation tasks whose decisive cue disappears from view long before it is needed. We study a lightweight alternative to architectural memory: an event-driven pipeline that detects when something disappears from the scene, describes only the disappeared content with a small vision-language model, and appends the resulting short text notes to the instruction of an otherwise unmodified $\pi_{0.5}$ policy. On the long-horizon RememberColor tasks of the MIKASA-Robo-VLA benchmark, the notes raise the success rate of the same policy, trained on the same demonstrations for the same number of steps, from 0.14 to 0.96 and 0.94 with three and five candidate colors and from 0.08 to 0.49 with nine. We decompose the remaining gap to an oracle note (0.87) along three axes. Selection: writing only what disappeared removes distractor notes, and changing the note writer of a fixed policy at test time changes its success by up to 24 points. Representation: naming colors from measured pixels instead of free-form captions reduces confusions between similar colors from 13 to 5 of 50 episodes. Utilization: in the remaining failures the note is correct, yet the policy approaches the wrong object. Across 26 long-horizon memory tasks, the notes help single-cue recall of colors and shapes but not tasks that require multiple cues, object tracking, counting, or timing, which delineates where lightweight language memory is and is not sufficient for VLA policies.

Keywords: Long-horizon manipulation, memory, partial observability, robot learning, vision-language-action models.

1. INTRODUCTION

Many manipulation tasks are partially observable in time: the information that determines the correct action is shown once and is no longer visible when the action must be taken. A robot that is shown a colored cube, waits while the cube is removed, and must later touch the cube of the same color among several candidates cannot solve the task from its current camera image alone. Vision-language-action (VLA) policies such as $\pi_{0.5}$ [1] are trained to map the current observation and a language instruction to actions, and they have no mechanism to carry such information across hundreds of control steps. On the 26 long-horizon memory tasks we study, a $\pi_{0.5}$ policy fine-tuned without memory reaches a mean success rate of only 0.05.

Recent work adds memory to VLA policies in several forms: latent memory banks [9, 10], recurrent memory tokens inside the backbone [7], visual keyframes selected when the action plan changes [6], and histories of past frames [11]. These approaches change what the policy sees, but it remains difficult to tell *why* a memory-augmented policy fails: because the relevant information was never written, because it was written in a form the policy cannot use, or because the policy does not act on correct information. Language memory makes these questions measurable. A text note can be read, replaced by the ground truth, or edited at test time without retraining, so each stage can be tested in isolation. RoboMME [8] observed that VLM-written language subgoals reach only about a third of the success obtained with ground-truth subgoals; our goal is to understand and reduce such a gap.

We present an event-driven language memory for VLA policies (Fig. 1). A detector fires when a part of the scene changes while the robot is still, the changed region is cropped and described by a small vision-language model, and only the content that *disappeared* is kept as a short note such as `Gone: a aqua cube.` The notes are appended to the instruction string; the policy architecture, inputs, and training recipe are unchanged. We analyze the pipeline along three axes: *selection* (when and what to write), *representation* (in which words), and *utilization* (whether the policy acts on the note).

Our contributions are:

- An event-driven semantic memory pipeline that preserves task-relevant information disappearing from view, without modifying the VLA architecture.
- Controlled evidence that selective writing and normalized attribute names change downstream decisions, including test-time note swaps on a fixed pol-

[Manuscript received ...; affiliations; e-mails.]

* Corresponding author.

©ICROS, KIEE and Springer 2026

icy, and that redundant or inconsistent notes degrade the policy.
- An analysis of the gap between memory correctness and action success using generated notes, oracle notes, content-swapped notes, and visual memory.
- An evaluation on 26 long-horizon memory tasks with one joint policy per method, characterizing where lightweight language memory helps and where it does not.

2. RELATED WORK

2.1. Memory-augmented VLA policies

MemoryVLA [9] and MemoryVLA++ [10] maintain perceptual and cognitive memory banks queried by a diffusion action expert. μVLA [7] adds recurrent memory tokens to a pretrained VLA backbone. SimpleMemVLA [11] provides a timestamped visual history to a video-capable backbone. Non-VLA visuomotor policies with memory include GMP [12] and Chameleon [13]. These methods report results on the standard MIKASA-Robo RememberColor tasks [4] with different backbones, training sets, and checkpoint selection; we therefore compare against our own baselines under identical conditions and list published numbers only for reference.

2.2. Event-driven memory selection

RePlanMem-VLA [6] writes a visual keyframe when overlapping action-chunk predictions disagree and evaluates the detection and retention of memory-relevant frames on MIKASA-Robo-VLA. Our trigger is an observation change rather than a plan change, and we store a text description of the removed content rather than the frame. We reimplement keyframe memory with our trigger as a baseline (Section 4).

2.3. Language memory for robot policies

RoboMME [8] benchmarks fourteen memory variants on top of $\pi_{0.5}$ and finds that language subgoals are strong when accurate (84% with oracle subgoals) but weak when generated by a VLM (33%). We study the same kind of gap for event notes and attribute it to selection, representation, and utilization.

2.4. Benchmarks

MIKASA-Robo [4] and its language-conditioned extension MIKASA-Robo-VLA [5] provide 90 manipulation tasks grouped by memory type, with 250 demonstrations per task. The reference $\pi_{0.5}$ policy of [5], fine-tuned on 14 tasks, reaches a mean success of 0.211.

3. METHOD

The policy is $\pi_{0.5}$ [1] fine-tuned with LeRobot [2]. Its observations are a top and a wrist camera image (128×128) and the proprioceptive state, and memory enters only through the instruction string. At every step the policy receives

`{instruction} Notes: (1)` n_1 `(2)` n_2 `...`

where $n_1, n_2, \ldots$ are the notes written so far. The same pipeline is applied to the demonstrations to build the training instructions and runs online during evaluation.

3.1. Event detection

Let I_t be the top-camera image and q_t the proprioceptive state. Step t is an event when (i) the robot has been still for the last $W = 5$ steps, i.e. $\|q_s - q_{s-1}\|_\infty < 10^{-3}$ for $s \in (t-W, t]$; (ii) at least $N = 8$ pixels changed by more than $\delta = 40$ intensity levels with respect to I_{t-1}; and (iii) this number exceeds $\rho = 4$ times its mean over the previous W steps. Condition (i) suppresses changes caused by the robot itself. The note is written once the scene has settled. The rule uses no task information. At test time, policy outputs below a small dead band are treated as still, which reproduces the event structure of the demonstrations.

3.2. Change localization

The largest connected component of changed pixels defines the event region. A square crop around it is taken from the frames before and after the event and upsampled to 384×384 with nearest-neighbor interpolation. A cue cube that covers 3×3 pixels of the 128×128 camera becomes legible to the captioner, and unchanged parts of the scene, including the robot, cannot enter the note.

3.3. Captioning and selection

SmolVLM2-2.2B [3] captions the before and after crops separately with the prompt *"List the objects in the image with precise color names, e.g. 'a cyan cube, a maroon cube'. The wooden surface is not an object. If there are no objects, answer 'nothing'."* Captioning the two crops jointly was unreliable about which content appeared and which disappeared. Content that appeared remains visible to the policy and is therefore not stored. Only the before side is kept, as `Gone: X.`, and notes whose X is empty or the table surface are dropped. A scene note of the full first frame is written once at $t = 0$.

3.4. Representation

The captioner identifies color families reliably but uses one name for pairs such as maroon and red, orange and yellow, or teal and cyan. We therefore replace the first color word of a note by a measured name: the mean RGB of the 25% most saturated pixels of the removed region, mapped to the nearest of the 16 HTML basic colors plus orange. A coarse, standard vocabulary keeps the measured name stable between the demonstrations and the online rollouts; a finer vocabulary (148 CSS colors) produced different names for the same cue under small rendering

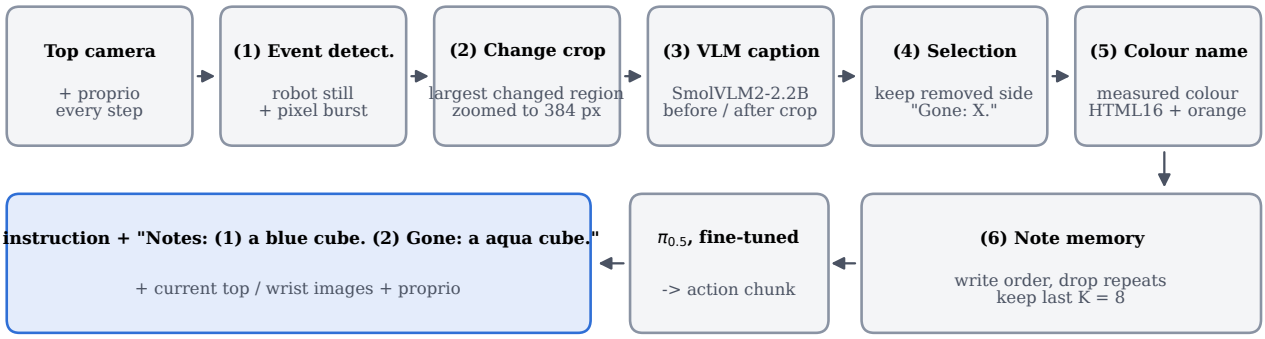


Fig. 1. Event-driven language memory. Notes are written only at change events, only for the content that disappeared, and are appended to the instruction of an unmodified $\pi_{0.5}$ policy. The same pipeline labels the training demonstrations and runs online at test time.



Fig. 2. Notes written for a RememberColor9-Long demonstration (cue: aqua cube). The change between $t = 99$ and $t = 101$ triggers an event; the zoomed crops show the removed cue. v3: `Change: a cyan cube, a maroon cube -> brown.` (the captioner splits the shaded faces into two objects); v4: `Gone: a cyan cube, a maroon cube.`; v6: `Gone: a aqua cube.` At the decision the v6 policy receives `Observe the cube's color, wait, then touch the cube of the same color. Notes: (1) a blue cube. (2) Gone: a aqua cube.`

differences. Only the first object of a note is kept, because the captioner occasionally describes the shaded top and side faces of one cube as two objects (Fig. 2).

3.5. Note memory and training

Notes are kept in write order. A note identical to the previous one is dropped, and the last $K = 8$ notes are appended to the instruction. For training, the instruction of every demonstration frame contains the notes written up to that frame, and $\pi_{0.5}$ is fine-tuned on these instructions with the standard objective and hyperparameters.

4. EXPERIMENTAL SETUP

4.1. Tasks

We use the *Long* variants of MIKASA-Robo-VLA [5], in which the cue is followed by a delay of 50–450 steps before the decision. The main analysis uses RememberColor3/5/9-Long (RC3/5/9-L): a cue cube is shown, removed, and after the delay 3, 5, or 9 candidate cubes appear; the robot must touch the one with the cue color. The generality study uses all 26 environments whose names carry the `-Long` suffix (Table 4), which cover object, capacity, sequential, tracking, and temporal memory. This is a custom subset: by the official split, 14 of them belong to the long and 12 to the medium split, and the official long tasks BatteriesChecker, TraceShapeSeq, and GatherAndRecall are not included.

4.2. Physics backend

The demonstrations of these tasks were collected by a motion planner with the CPU physics backend. Replaying the recorded actions with the original seeds succeeds in 8/8 episodes on the CPU backend and 0/8 on the GPU backend used by the canonical evaluation, with trajectory deviations of 2.5–4.5 cm caused by different inverse-kinematics solvers. We therefore evaluate on the CPU backend, consistent with the data.

4.3. Training and evaluation

Table 1 summarizes the protocol. One policy is trained jointly per task set and method. We report success rates with Wilson 95% confidence intervals. For the RememberColor tasks we additionally record which candidate cube the gripper approached most closely, which separates the choice of the target from the execution of the touch.

Table 1. Training and evaluation protocol (identical for all methods).

Policy	$\pi_{0.5}$ (`pi05_base`), LeRobot, bf16
Inputs	top + wrist RGB 128 × 128, proprioception
Actions	chunk 10, execute 5, end-effector delta pose
Data	250 demonstrations per task
Training	joint over a task set, batch 16 RC-L (3 tasks): 20k steps; Long-26: 90k steps
Evaluation	50 episodes per task, fixed seeds RC9-L: 150 episodes for the main models
Checkpoint	last checkpoint (no best-checkpoint selection)
Physics	CPU backend (matches demonstrations)
Metrics	success; for RC tasks also the approached cube

4.4. Compared methods

All methods share data, training steps, and evaluation; only the memory differs.

- **No memory**: the original instruction.
- **Notes v2–v6**: successive versions of our pipeline (Table 2). v2 captions the full frame with SmolVLM2-500M at each event; v3 captions zoomed before/after crops as `Change: X -> Y.`; v4 keeps only the removed side (selection); v5 and v6 add measured color names with a fine (148) and a coarse (17) vocabulary (representation).
- **v7**: v6 without the scene note at $t = 0$.
- **Periodic captions**: a full-frame scene caption every 50 steps with the same captioner, without event detection or selection.
- **Visual memory**: the zoomed crops (*crop*) or full frames (*keyframe*, as in RePlanMem-VLA [6]) written at our events are tiled into an additional camera image.
- **Oracle note**: the true cue color name appended to the instruction (upper bound for text memory).

5. RESULTS AND ANALYSIS

5.1. Main result on RememberColor-Long

Table 2 shows that event notes turn tasks the policy cannot solve into tasks it solves almost perfectly with three and five colors (0.14 to 0.96 and 0.94), and raise the nine-color task from 0.08 to 0.49; the confidence intervals do not overlap. The policy without memory performs at chance (1/3, 1/5, 1/9 for an always-reaching policy, lower when it does not touch). The visual crop memory, which contains the same cue image, stays at the no-memory level: inspection of the online memory image confirmed that it was written correctly, so the policy ignores the additional camera input under our data and training budget. The note versions v2, v4, and v6 are statistically indistinguishable on RC9-L; we therefore analyze how the versions change the *types* of errors rather than claiming that a later version is better.

Table 2. Success rate on RememberColor-Long (one joint policy per row, 20k steps, 50 episodes per task; RC9-L also with 150 episodes and Wilson 95% CI).

Memory	RC3-L	RC5-L	RC9-L	RC9-L (n=150)
None	0.14	0.14	0.06	0.08 [0.05, 0.13]
Visual (crop)	0.12	0.12	0.14	–
Notes v2	0.86	0.86	0.50	0.50 [0.42, 0.58]
Notes v3	0.96	0.84	0.38	0.41 [0.33, 0.49]
Notes v4	0.94	0.90	0.54	0.49 [0.41, 0.57]
Notes v5	1.00	0.92	0.50	–
Notes v6	0.96	0.94	0.48	0.49 [0.41, 0.57]
Oracle note	0.96	0.98	0.80	0.87 [0.80, 0.91]

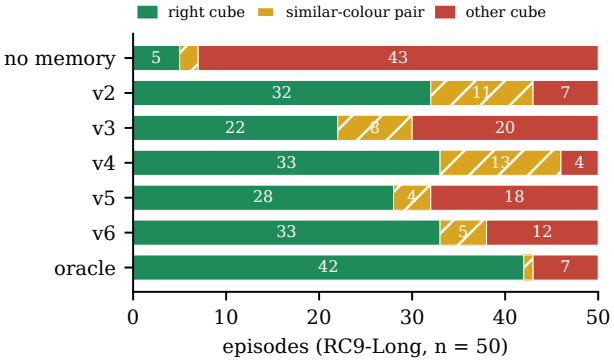


Fig. 3. Which cube the policy approached on RC9-L (50 episodes): the cue color, the other member of a pair the captioner names alike (red/maroon, yellow/orange, cyan/teal), or another cube.

Since the delay before the decision is drawn per episode, we also binned the episodes by delay (50–183, 184–316, 317–450 steps). The success of the note policy does not depend on the delay within the trained range (RC9-L v6: 0.48, 0.42, 0.57; oracle: 0.86, 0.87, 0.87), as expected from a memory that is written once and kept. With a fixed delay of 1000 steps, more than twice the longest delay seen in training, the note policy keeps its success on RC5-L (0.68 to 0.76) while the policy without memory stays near chance (0.08); both at 45k steps on the 26-task mixture. [RC9-L and the other memory types.]

5.2. Selection

v3 notes record both sides of every change. When the candidates appear, the note lists all nine candidate colors (`Change: a wooden surface -> a red cube, a green cube, ...`), so the instruction contains every color word at the moment of the decision. Removing this appeared content (v4) reduces approaches to unrelated cubes from 20 to 4 of 50 (Fig. 3). The effect is causal for a fixed policy (Table 3): adding the appeared content at test time to the v4 policy lowers its success from 0.54 to

Table 3. Test-time note swaps on RC9-L (50 episodes): the policy is fixed and only the note writer changes. Right / pair / other as in Fig. 3. †Every online note names another candidate cube; ‡episodes that reached the cube named by the wrong note.

Trained with	Tested with	Success	Right / pair / other
v4	v4	0.54	33 / 13 / 4
v4	v3 (+ appeared content)	0.30	17 / 9 / 24
v3	v3	0.38	22 / 8 / 20
v3	v4 (removes it)	0.46	27 / 9 / 14
v6	no notes	0.08	8 / 2 / 40
v6	wrong cue color†	0.34	19 / 14‡ / 17
v6	true cue color	[C1]	[]

Table 4. Success on the 26 `-Long` tasks (one joint policy per method, 50 episodes per task). Both policies at 45k steps of the same 90k schedule. [Add v7, periodic, keyframe columns; 90k rows.]

Family (memory type)	Variants	No memory	Notes v6
RememberColor (object)	3 / 5 / 9	.22 / .06 / .04	.74 / .68 / .28
RememberShape (object)	3 / 5 / 9	.20 / .12 / .16	.38 / .26 / .16
ShapeAndColor (object)	3x2 / 3x3 / 5x3	.08 / .06 / .12	.32 / .26 / .18
BunchOfColors (capacity)	3 / 5 / 7	.00 / .00 / .00	.00 / .00 / .00
SeqOfColors (capacity)	3 / 5 / 7	.00 / .00 / .00	.00 / .00 / .00
ChainOfColors (sequential)	3 / 5 / 7	.00 / .00 / .00	.00 / .00 / .00
ShellGame (tracking)	Touch / Lamp	.20 / .14	.16 / .26
BlinkCount (temporal)	E / M / H	.02 / .00 / .00	.06 / .04 / .00
TimedTransfer (temporal)	E / M / H	.00 / .04 / .06	.00 / .00 / .00
Mean (26 tasks)		0.058	0.145

0.30, and removing it from the v3 policy raises its success from 0.38 to 0.46. Since v3 and v4 also differ in sentence form, the swap measures the joint effect of content and format. [Periodic captions on the 26 tasks (C2): effect of event-triggered writing versus writing at fixed intervals.]

5.3. Representation

With v4, 13 of 50 episodes end at the cube that forms a similar-color pair with the cue; both captioners call maroon "red" and orange "yellow", so the note cannot separate them. Measured color names (v6) reduce these confusions to 5. The finer CSS vocabulary (v5) also removed pair confusions but introduced different names for the same color in the demonstrations and in online rollouts (for example "blue" and "mediumblue"), and approaches to unrelated cubes rose to 18. A representation must therefore be both discriminative for the task and stable between training and deployment.

5.4. Utilization

v6 reduces pair confusions but not the total error. In all 17 v6 episodes in which the policy approached a wrong cube, the cue note named the correct color. Most of these errors are among blue, aqua, and teal, and in these episodes the scene note at $t = 0$ describes the cue as "a blue cube" while the measured note says "aqua" or "teal". The policy thus receives two inconsistent descriptions of the same object. [v7 without the scene note (C4).] The oracle note, which names the cue exactly, reaches 0.87 on RC9-L; in its 50 standard episodes the policy still approached an unrelated cube seven times, and in two further episodes it approached the right cube without completing the touch. Together with the visual memory that is written correctly but ignored, these results indicate that, once the note is correct, the remaining bottleneck lies in grounding the remembered attribute to the current candidates. The content-swap control confirms that the policy reads the note. When the color word in every online `Gone:` note is replaced by the color of another candidate cube present in the scene, success drops from 0.96 / 0.94 / 0.48 to 0.44 / 0.64 / 0.34 on RC3 / RC5 / RC9-L, and the robot goes to the cube named by the wrong note: on RC3 and RC5 every episode that does not reach the right cube reaches the note's cube (22 of 22 and 15 of 15), and on RC9 14 of 31. The scene note at $t = 0$ is left unchanged and still names the true cue, which explains why some episodes still reach the right cube. With the notes removed, the same policy falls to 0.26 / 0.10 / 0.08, close to the policy trained without notes (0.14 / 0.14 / 0.06). [true-color row]

5.5. Memory types across 26 long-horizon tasks

Table 4 reports one policy trained jointly on all 26 tasks with and without notes. Without memory, $\pi_{0.5}$ reaches 0.058 on average (0.048 after the full 90k steps); with notes it reaches 0.145. Notes help all nine single-cue object tasks: colors, shapes, and their combinations (mean 0.36 versus 0.12 at equal steps; higher on eight of nine and equal on RememberShape9). On the other 17 tasks both policies stay near zero (0.03 each). They do not help tasks that require several cues at once (BunchOfColors, SeqOfColors) or in order (ChainOfColors). For ChainOfColors3 we had verified separately that even the true color sequence in the instruction yields 0/50, so that family is limited by the multi-touch skill rather than by memory. In tracking tasks (ShellGame) the shuffle is a continuous motion of objects that remain visible, which our event trigger does not describe; in BlinkCount each blink produces a note but the window of eight notes truncates long sequences; and in TimedTransfer the frequent lamp and object changes produce about 97 events per episode, most of which are noise. [Comparison of the five memory types (no memory, v6, v7, periodic captions, keyframes) at 45k steps on all 26 tasks.]

5.6. Discussion and limitations

Event notes are effective when a single attribute of a single object must be carried across a long delay, and they make the failure modes of a memory-augmented

policy observable. The measured color names are specific to color attributes and are not a general representation. The 26-task results come from one training seed per method; for the main RememberColor comparison the confidence intervals are reported over 150 episodes. Memory that must track continuous motion, count, or measure elapsed time requires structured state rather than event descriptions, which we leave to future work, together with object-centric and learned retrieval memories. [Interference experiment (C) and event-selection analysis (D) if completed.]

6. CONCLUSION

We showed that a VLA policy can solve long-horizon memory tasks it otherwise cannot, without architectural changes, by appending short notes about what disappeared from the scene. Selecting only removed content and naming attributes in a stable vocabulary are necessary for this, but not sufficient: with correct notes, a remaining gap lies in how the policy grounds the remembered attribute in the current scene. Text memory makes each of these stages measurable, which we hope will help design memories that go beyond single-cue recall.

DECLARATIONS

Conflict of Interest

The authors declare that there is no competing financial interest or personal relationship that could have appeared to influence the work reported in this paper.

Authors' Contributions

[]

Funding

[]

Data Availability

[Code, note datasets, and evaluation logs: repository link.]

Use of AI Tools

[The authors used [tool] for [role]. The authors remain fully responsible for the content of the manuscript.]

REFERENCES

[1] Physical Intelligence [K. Black et al.], "$\pi_{0.5}$: A vision-language-action model with open-world generalization," *arXiv preprint* arXiv:2504.16054, 2025.

[2] R. Cadene [et al.], "LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch," [Online]. Available: https://github.com/huggingface/lerobot, 2024.

[3] A. Marafioti [et al.], "SmolVLM: Redefining small and efficient multimodal models," *arXiv preprint* arXiv:2504.05299, 2025.

[4] E. Cherepanov [et al.], "Memory, benchmark & robots: A benchmark for solving complex tasks with reinforcement learning," *Proc. of International Conference on Learning Representations (ICLR)*, 2026.

[5] [Authors], "MIKASA-Robo-VLA: Benchmarking memory in VLA models for long-horizon manipulation," *arXiv preprint* arXiv:2610.00604, 2026.

[6] [Authors], "When plans change, remember: Replanning-guided visual memory for vision-language-action models," *arXiv preprint* arXiv:2610.22556, 2026.

[7] E. Cherepanov [et al.], "μVLA: On recurrent memory for partially observable manipulation in VLA models," *arXiv preprint* arXiv:2606.12497, 2026.

[8] [Y. Dai et al.], "RoboMME: Benchmarking and understanding memory for robotic generalist policies," *Proceedings of Machine Learning Research*, vol. 306, [pages], 2026.

[9] [Authors], "MemoryVLA: Perceptual-cognitive memory in vision-language-action models for robotic manipulation," *Proc. of International Conference on Learning Representations (ICLR)*, 2026, arXiv:2508.19236.

[10] [Authors], "MemoryVLA++: Temporal modeling via memory and imagination in vision-language-action models," *arXiv preprint* arXiv:2606.09827, 2026.

[11] [Authors], "SimpleMemVLA," *arXiv preprint* arXiv:2609.05533, 2026. [full title]

[12] [Gao, Liu, Li, Song], "Gated memory policy: In-context memorization and adaptation," *arXiv preprint* arXiv:2604.18933, 2026.

[13] [Authors], "Chameleon: Control-indexed prospective memory for visuomotor manipulation," *arXiv preprint* arXiv:2603.24576, 2026.